



Algorithm & C-Programming

Mock exam:
Bachelor B1 (School year 2028)

The use of calculators and any electronic device is prohibited.

If, during the exam, a candidate notices what appears to be an error in the wording, they must mention it in their answer sheet and continue the exam, explaining the reasoning behind any decisions they choose to make.



Infos

Edition:

1.0

 (2025.12.10)

svandamme@omnesintervenant.com



00. FOREWORD

A. General information

Quick reminder: it is always worth reading the entire wording before you start.
Identify where you can earn points efficiently and quickly.

It is highly recommended to group the questions of the same exercise together, instead of jumping back and forth between exercises.

Keep a clear structure using the exercise headers, and leave some space for possible corrections later.
Example:

Exercise 1

Q. 1) Answer 1, blabla (...)

Q. 2) // I don't know the answer.

Q. 3) (space for later)

Q. 4) Answer 4, blabla (...)

Even if you cannot answer a question immediately, leave room for it later — like Q.3 in the example.

Grading scales may be slightly adjusted during correction.

This exam was designed to be about: **1h30** long.
Good luck with the exam. :)

B. Prerequisite chapters

For this exam, the following chapters are relevant:

Basic of C programming (variable, type...)

Arrays and matrixes

Pointers

Debugging/Common errors

Control flow

Algorithms reasoning

Functions



01 – Mystery function (20/100 pts)

You're given the following function:

```
int mystery(int a, int b)
{
    int x = 0;

    while (a < b)
    {
        if (a % 2 == 0)
            a = a * 3 + 1;
        else
            a = a + 2;

        x++;
    }

    return x;
}
```

Q.1) Explain the principles of this algorithm.

What does the function compute?

What is its purpose?

You may illustrate your explanation using examples, diagrams, or small computations.

Q.2) What is the result of the function call: `mystery(5, 20)` ;



02 – Spot the mistakes (15/100 pts)

For each code snippet, find the errors that prevent the program from compiling **or** giving the correct result.

Q.1) First code snippet.

```
void swap(int a, int b)
{
    int t = a;
    a = b;
    b = t;
}

int main()
{
    int x = 3, y = 7;
    swap(&x, &y);
    printf("%d %d\n", x, y);
}
```

Q.2) Second code snippet.

```
void countDown(int n)
{
    for (int i = n; i >= 0; i++)
        printf("%d\n", n);
}

int main()
{
    int v = 5;
    countDown(v);
    return 0;
}
```

Q.3) Find the error for this code.

```
float average(int a, int b, int c)
{
    return a + b + c / 3;
}

int main()
{
    int x, y, z;
    printf("Enter 3 scores: ");
    scanf("%f %f %f", &x, &y, &z);

    float m = average(x, y, z);
    printf("Average = %d\n", m);

    return 0;
}
```



03. Value tracking & pointers (15/100 pts)

You are given the following short program.

Your goal is to understand how values evolve, what is modified, and how pointers influence each variable.

```
int main()
{
    int A = 5, B = 12, C = 0;
    int *p = &A;

    C = B - A;
    B = C + 3;

    printf("Before update(): A = %d, B = %d, C = %d\n", A, B, C);

    update(p, C);

    C = A + B;
    p = &C;
    *p = *p + 4;

    printf("After update(): A = %d, B = %d, C = %d\n", A, B, C);

    return 0;
}
```

```
void update(int *u, int v)
{
    int t = *u + v;
    *u = v - 1;
    v = t + 2;

    printf("Inside update(): *u = %d, v = %d, t = %d\n", *u, v, t);
}
```

Q.1) Write exactly what the program 3 *printf(...)*, in the correct order.

Q.2) Draw a small, clear diagram of the variables and pointers:

- Before the call to `update()`, after the line (2)
- After the call to `update()`, just before the final `printf`

Make sure your diagrams show:

- the current values of A, B, and C



- where the pointer `p` points at each moment

Q.3) Explain in 2–4 lines how the parameter passing in `update()` works:

- Which variable is modified through the pointer?
- Why does changing `v` inside `update()` not affect any variable in `main()`?



04. Matrix Column Sum (25/100 pts)

You are given an $M \times N$ matrix of integers ($\forall (M, N) \in (\mathbb{N}^*)^2$).

Write a function that returns the **sum of a specific column** of a matrix.

Example for a 3×4 :

1	2	3	4
1	0	5	9
10	7	2	3

If the user asks for column 2 (0-indexed), the sum is:

1	2	3	4
1	0	5	9
10	7	2	3

columnSum = $3 + 5 + 2 = 10$

The value included in the sum are related to the selected **COLUMN**.

Then, you add up all the values from all rows in the column.

Q.1) Create a function:

```
int columnSum(int matrix[][N], int m, int n)
{
    ...
}
```

Q.2) Write a main() function that:

1. Defines a matrix of your choice (for example a 3×4 matrix as shown in the example).
2. Asks the user to input a column index col.
3. Displays the result using printf() in the following format:

```
printf("The sum of column %i is: %i", col, columnSum(...)) ;
```



05. Palindrome (25/100 pts)

A *palindrome* is a word that reads the same forwards and backwards.

Examples:

"radar", "level", "noon"

Write a program that asks the user to enter a word (**without spaces**), and checks whether it is a palindrome.

Q.1) Write the function:

```
int isPalindrome( char word[] )  
{  
    ...  
}
```

The function must:

- return **1** if the string is a palindrome
- return **0** otherwise

Compare characters from the start and end, moving toward the center.

Ignore no special characters (basic version).

Q.2) Write a main() that:

1. asks the user to enter a word (max 50 characters)
2. calls "isPalindrome()"
3. prints either:
"Palindrome OK" or "Not a palindrome"